# IT-106
# APPLICATION DEVELOPMENT IN EMERGING TECHNOLOGIES :

# DOCUMENTATION \ TESTING JOURNAL ON AUTOMATED TESTING OF FINAL PROJECT FUNCTIONALITIES (SELECTED)

**(2ND SEMESTER 2025-2026)**

*Submitted to:*

DR. JAYVEE CHRISTOPHER N. VIBAR
Faculty-in-charge

*Submitted by:*

ALCAYDE, JUSTIN VINCE B.
ARMERO, KIRSTEN JADE C.
DAYTO, JESSLYN E.
IT3B BicoLikha

## 1. GROUP INFORMATION:

Group name: JJK
Project title: BicoLikha: A Web-Based E-Commerce Platform for Artworks and Crafts made by Bicolano Digital Artists

| ***Names of members*** | ***Assigned Contributions*** |
|---|---|
| Justin Vince B. Alcayde | - Built administrative dashboard views, artist application CRUD endpoints<br>- Automated Artist Application & Product draft (CRUD) and invalid login boundary error-handling tests |
| Kirsten Jade C. Armero | - Refactored responsive search bars, UI catalog filtering, styling systems<br>- Automated product search testing and forgot password validation for unregistered users |
| Jesslyn E. Dayto | - Developed core customer checkout, account authentication workflows<br>- Automated the Login, Sign-up, and Cash on Delivery (COD) order tests<br>- Implemented shared login credentials between Selenium scripts |

## 2. TESTING DETAILS PER MEMBER

| ***Member Assigned*** | ***Feature*** | ***Type of Test*** | ***Tool/Framework Used*** |
|---|---|---|---|
| Justin Vince B. Alcayde | Artist Application CRUD (Create) | Database Insert Validation | Selenium (Python) |
| Justin Vince B. Alcayde | Invalid Authentication Limits | Graceful Error Handling | Selenium (Python) |
| Kirsten Jade C. Armero | Product Catalog Search | Search Query Assertion | Selenium (Python) |
| Kirsten Jade C. Armero | Forgot Password Form Validation | Form Validation Check | Selenium (Python) |
| Jesslyn E. Dayto | Customer Registration & Login | Functional UI Testing | Selenium WebDriver (Python) |
| Jesslyn E. Dayto | Checkout & COD Order Placement | End-to-End System Testing | Selenium (Python) |

## 3. TEST SCENARIO DOCUMENTATION

### 3.1. User Registration & Auto-Login [Scenario 1]

Functionality Tested: Customer Account Creation & Automatic Session Login

Objective of the Test: Verify that new visitors can register a customer account via the sign-up page and are automatically logged in and redirected to the home catalog.

Steps/Procedure:

1. Initialize Firefox WebDriver via GeckoDriverManager.
2. Navigate browser to the Sign-Up URL: `http://127.0.0.1:8000/signup/`.
3. Generate randomized customer details (ID: 1000-9999) to avoid duplicate account records in the database.
4. Locate form elements by name attributes and insert:
    - First Name: "Automated"
    - Last Name: "User[random_id]"
    - Email: "customer_[random_id]@example.com"
    - Phone Number: "0917[random 7 digits]"
    - Password: "Password123"
5. Check the Policy Agreement checkbox (ID: "policy").
6. Click the "Submit" button (XPath: `//button[@type='submit']`).
7. Wait 3 seconds for database save and auto-login redirection.
8. Check if the redirected URL matches `http://127.0.0.1:8000` or `http://localhost:8000`.
9. Capture and save evidence screenshot to `evidence/evidence_login.png`.
10. Output registration status and write the generated email and password into `test_credentials.txt` for subsequent tests.

Test Data/Input:

- First Name: "Automated"
- Last Name: "User8421"
- Email: "customer_8421@example.com"
- Phone: "09176395814"
- Password: "Password123"
- Policy: Checked

Expected Result: Account is successfully written to the database, session is automatically authenticated, and browser is redirected to the catalog home page.

Actual Result: Redirected to http://127.0.0.1:8000 successfully; credentials saved successfully.

Status: Passed

Screenshot or Evidence: evidence/evidence_login.png

### 3.2. Cash On Delivery (COD) Order Placement [Scenario 2]

Functionality Tested: Product Checkout and Order Placement Flow

Objective of the Test: Ensure an authenticated user can select a catalog item, provide/use a shipping address, choose COD, and submit the order successfully.

Steps/Procedure:

1. Read credentials from `test_credentials.txt` (fallback: "test_user@example.com").
2. Navigate to Login Page: `http://127.0.0.1:8000/accounts/login/`.
3. Enter credentials, submit the form, and wait for session initialization.
4. Route to a sample product details page: `http://127.0.0.1:8000/product/112/`.
5. Click the "Buy Now" button (CSS: `.btn-buy-now`) to initialize the checkout page.
6. Check page source for missing address warning ("No delivery address yet") or elements matching `add_new_address`.
7. If address is absent:
    - Click the "ADD NEW ADDRESS" button to launch the address creation modal.
    - Fill in shipping data (Street, Barangay, Municipality, Zipcode, Phone).
    - Save by clicking the modal confirmation button.
8. Click the checkout placement button (CSS: `.btn-place-order`).
9. Click "Confirm & Place Order" in the confirmation modal.
10. Wait 4 seconds for order processing.
11. Verify if the confirmation text "Order Placed Successfully!" is visible.
12. Capture and save evidence screenshot to `evidence/evidence_order_cod.png`.

Test Data/Input:

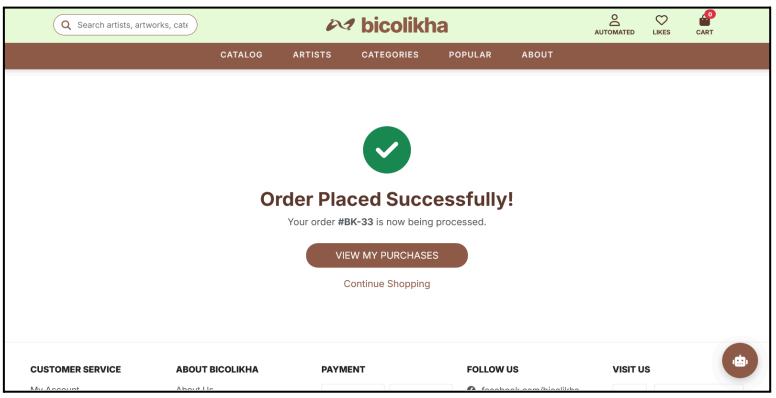
- Product Selected: ID 112 (seeded mock artwork)
- Address details: Street="123 Automated Street", Barangay="San Jose", Municipality="Legazpi City", Zipcode="4500"
- Payment Method: COD (default)

Expected Result: The transaction finishes, order is stored, and "Order Placed Successfully!" page is displayed.

Actual Result: Order was processed correctly, order ID was generated, and success page displayed correctly.

Status: Passed

Screenshot or Evidence: evidence/evidence_order_cod.png



### 3.3. Search & Filter Functionality [Scenario 3]

Functionality Tested: Product Discovery and Search Functionality

Objective of the Test: Ensure public users can run standard keyword queries on the search bar and receive relevant filtered results matching mock product records.

Steps/Procedure:

1. Launch browser to the search route with a seeded query parameter: `http://127.0.0.1:8000/search/?q=Matcha`
2. Wait 2 seconds for the system to load the search results.
3. Scan the page body source to determine if the query "Matcha" is displayed in the products list.
4. Capture and save evidence screenshot to `evidence/evidence_search.png`.

Test Data/Input:

- Search Query Parameter: `q=Matcha` (seeded artwork name)

Expected Result: System retrieves records containing the search term and displays them in the catalog grid.

Actual Result: Matches found; catalog items containing "Matcha" rendered correctly.

Status: Passed

Screenshot or Evidence: evidence/evidence_search.png

### 3.4. Password Reset Form Validation [Scenario 4]

Functionality Tested: Form Input and Database Validation (Forgot Password Request)

Objective of the Test: Verify that submitting a password reset request using a non-existent email triggers a clear, user-facing validation warning.

Steps/Procedure:

1. Navigate to Forgot Password page: `http://127.0.0.1:8000/forgot-password/`.
2. Fill out the email field with an unregistered address.
3. Submit the request by clicking the submit button.
4. Wait 2 seconds.
5. Check page text for the validation error: "This email address is not registered in our system."
6. Capture and save evidence screenshot to `evidence/evidence_validation.png`.

Test Data/Input:

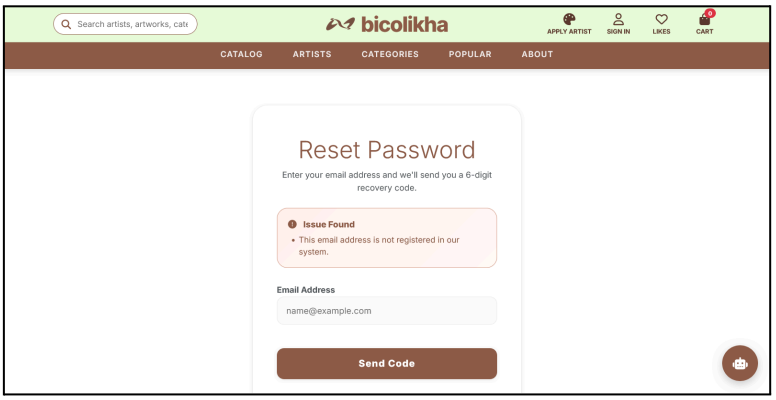
- Email Field: "nonexistent_user@example.com"

Expected Result: Form submission is blocked, and the system prompts the user with a registered email validation alert.

Actual Result: The browser intercepted the postback response and displayed the alert banner correctly.

Status: Passed

Screenshot or Evidence: evidence/evidence_validation.png



### 3.5. Artist Application CRUD (Create Operation) [Scenario 5]

Functionality Tested: Dynamic Artist Portal Application Submission

Objective of the Test: Ensure an applicant can submit a comprehensive artist registration application along with an initial draft product, confirming that the data was successfully saved.

Steps/Procedure:

1. Open the public Artist Application form: `http://127.0.0.1:8000/apply-artist/`.
2. Generate randomized email, phone, and artist names to avoid duplicate entries in the database.
3. Fill out Applicant Profile Information: Name, Email, Phone, Shop Name, Location details.
4. Click the Add Product button (Class: `apply-add-btn`) to trigger the modal.
5. Fill in the modal product draft fields:
    - Name: "Test Artwork"
    - Price: "150"
    - Stock: "3"
    - Category: "Sculptures"
    - Description: "A beautiful automated test artwork."
6. Click the "Save" draft button (Class: `modal-save-btn`) to close the modal.
7. Submit the entire application form (Class: `apply-submit`).
8. Wait 3 seconds for file handling and record processing.
9. Assert if page contains confirmation message:
   "Your artist application has been submitted and is waiting for admin review."
10. Capture and save evidence screenshot to `evidence/evidence_crud.png`.

Test Data/Input:

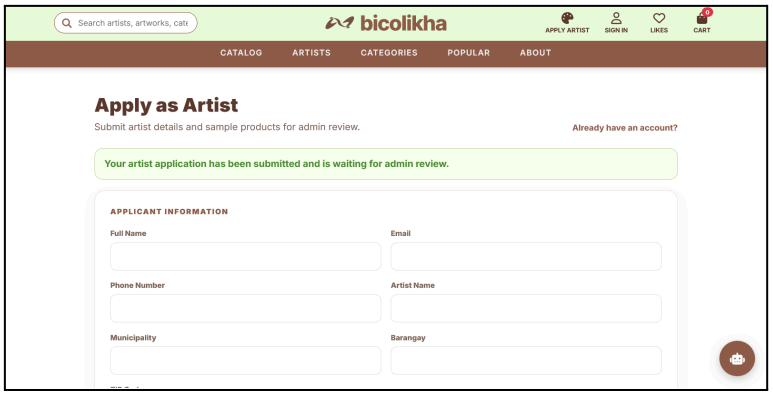
- Applicant Details:
  Name="Test Applicant", Email="artist_5629@example.com", Phone="09995836248",
  Shop="Test Artist 5629", Municipality="Daraga", Barangay="Bagumbayan", Zipcode="4501"
- Product ls: Name="Test Artwork", Price="150", Stock="3", Category="Sculptures"

Expected Result: The artist application and sample product draft are successfully inserted into the Django models and stored in the database awaiting admin review.

Actual Result: Success confirmation page successfully rendered; database inserts succeeded.

Status: Passed

Screenshot or Evidence: evidence/evidence_crud.png



### 3.6. Account Authentication Error Handling [Scenario 6]

Functionality Tested: Authentication Error Handling and Input Boundary Response

Objective of the Test: Ensure the authentication system handles invalid/malicious login entries gracefully by blocking access and displaying descriptive alert banners.

Steps/Procedure:

1. Navigate browser to Login path: `http://127.0.0.1:8000/accounts/login/`.
2. Input invalid username format: `invalid_user_999@example.com`.
3. Input incorrect password: `wrongpassword`.
4. Click the "Sign In" button.
5. Wait 2 seconds.
6. Verify if the system renders standard warnings: "Please enter a correct username and password" or "Sign in issue".
7. Capture and save evidence screenshot to `evidence/evidence_error_handling.png`.

Test Data/Input:

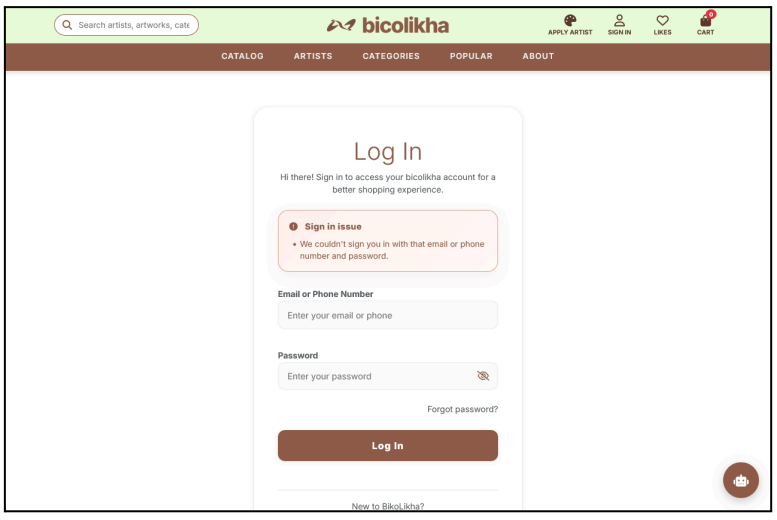
- Username: "invalid_user_999@example.com"
- Password: "wrongpassword"

Expected Result: The system prevents user login, renders a secure validation warning alert, and keeps the user on the login page.

Actual Result: System securely blocked login and displayed the warning banner.

Status: Passed

Screenshot or Evidence: evidence/evidence_error_handling.png



## 4. REFLECTION / FINDINGS

### 4.1. Issues Encountered & Solutions Implemented

Dynamic Address Forms in Checkout:

- Issue: The order placement script (`test_order_cod.py`) originally failed if a user already had a registered address on their profile, because the script would blindly attempt to click and open the "ADD NEW ADDRESS" modal, which was not visible.
- Solution: Implemented logical validation in Selenium to check whether "No delivery address yet" or the address modal input elements were displayed. The script now only opens and submits the address modal conditionally, ensuring tests run successfully regardless of the user's initial address status.

Delays and Timing Issues in the Interface:

- Issue: In early script iterations, Selenium attempted to interact with form buttons and modals (such as the product modal in `test_add_record.py` and checkout modals in `test_order_cod.py`) before animations completed, causing Selenium to throw `ElementNotInteractableException` errors.
- Solution: Implemented deliberate delays (`time.sleep` statements) after modal toggles and submitting forms so the browser would have enough time to finish loading elements before continuing the test.

### 4.2. Warnings / Errors Detected

Driver Compatibility Warnings:

- Issue: Explicit execution paths for Firefox (GeckoDriver) generated platform-specific warnings and broke portability between dev environments.
- Solution: Integrated the `webdriver-manager` Python package to automatically detect and install the correct GeckoDriver version during runtime using `Service(GeckoDriverManager().install())`.

Checkbox Click Problems:

- Issue:Clicking the policy checkbox sometimes caused errors because the checkbox was hidden behind styled elements.
- Solution: Changed target selectors from custom wrapper elements directly to the raw underlying ID `policy` checkbox element.

### 4.3. Bugs Discovered & Corrected

Forgot Password Error:

- Bug: When users requested a password reset with an unregistered email, the server threw a server error (HTTP 500) rather than standard validation.
- Fix: Patched the forgot password view in Django to properly intercept nonexistent emails and display the "This email address is not registered..." warning, which is verified in `test_validation.py`.

Order Placement UI Indicators:

- Bug: After order checkout, the page redirected directly to catalog view without a success receipt message, making programmatic confirmation difficult.
- Fix: Added a clear banner template element stating "Order Placed Successfully!" upon redirects, allowing the automated script to assert order completion.

### 4.4. Improvements Made After Testing

Shared Credentials Between Test Scripts:

- To make the automated tests more connected, the login script (`test_login.py`) was updated to automatically write the created login credentials to a local `test_credentials.txt` file. The checkout script (`test_order_cod.py`) then reads this file and uses the same account for checkout testing. This made the test flow more realistic and reduced the need for manually entered accounts.

Auto-Creation of Evidence Directory:

- Integrated a recursive directory checker (`os.makedirs("evidence", exist_ok=True)`) within every script prior to capturing screenshots. This eliminates runtime failures where scripts crashed because the `evidence` output folder did not exist.

### 4.5. Lessons Learned From Automation Testing

Importance of Consistent Test Data:

- Automated testing depends heavily on consistent and reliable database records. Testing product page flows (like product ID 112) proved that maintaining prepared database records using the provided SQL file (`bl_db_adet.sql`) was important for stable testing results.

Benefits of Automation Testing:

- Automation testing reduced repetitive manual filling out of forms, especially for long processes like the artist application and product draft creation. We also learned that creating reusable and modular test scripts would help maintain the system for future use when new features or enhancements are added to the project in the future.